EMBEDDED LINUX SYSTEMS

BÁO CÁO KỸ THUẬT

ASSIGNMENT 01

HOW TO WRITE A DEVICE DRIVER

Phạm vi: ioctl Character Driver và Counter Misc Device Driver, build ARMv7, kiểm thử thực tế trên QEMU vexpress-a9. Tài liệu tổng hợp từ source code và các file log do người thực hiện cung cấp.

THÔNG TIN	NỘI DUNG
Môn học	Embedded Linux Systems
Bài thực hành	Assignment 01 - Chapter 8, 9, 10
Sinh viên	[Họ và tên - bổ sung trước khi nộp]
Mã sinh viên	[MSSV - bổ sung trước khi nộp]
Nền tảng thử nghiệm	ARMv7 / QEMU vexpress-a9 / Linux Kernel 5.15.0
Ngày ghi nhận log	22/10/2026

Trạng thái chứng cứ: Hai bộ kiểm thử tự động đều kết thúc bằng thông báo **TEST FINISHED**; chi tiết từng trường hợp được trình bày trong các trang kết quả.

I. Tóm tắt và phạm vi

Assignment 01 gồm hai phần triển khai kernel module độc lập: **asgn1_driver** sử dụng character-device API truyền thống và **counter_driver** dựa trên miscdevice. Cả hai sử dụng giao diện tệp thiết bị trong Linux, nhưng phục vụ hai mục tiêu khác nhau.

1.1. Hạng mục đã triển khai

Thành phần	Điểm kỹ thuật đã có	Bằng chứng
Task 1: asgn1_driver	Character driver, 4 lệnh ioctl, mutex, read/write, header dùng chung	asgn1_driver.c; asgn1_ioctl.h; test_asgn1_output.txt
Task 2: counter_driver	misc_register, atomic_t, read tự tăng, write đặt lại, /proc/counter_info	counter_driver.c; test_counter_output.txt
Build + QEMU	Hai tệp module được tạo cho ARM EABI5; chạy trên Kernel 5.15.0 trong QEMU	Hai tệp .ko và log test

1.2. Phạm vi báo cáo

Báo cáo đi theo cấu trúc kỹ thuật: sơ đồ hệ thống, so sánh character/block/network device, phân tích lựa chọn API và đồng bộ, cuối cùng là kiểm thử có số liệu thực. **Không trình bày riêng Task 4 - bộ câu hỏi lý thuyết** theo phạm vi thực hiện đã chọn.

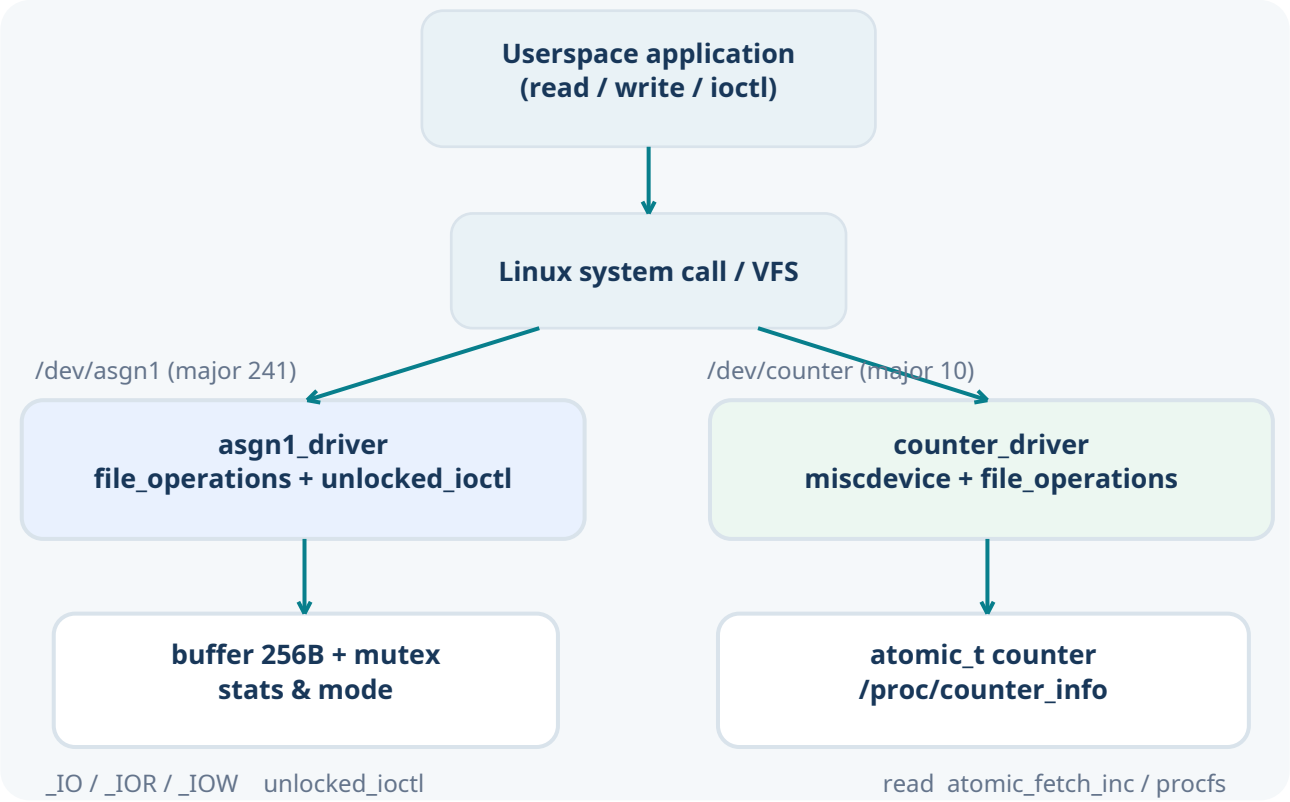
1.3. Cấu trúc nội dung

Mục	Nội dung
II–III	Kiến trúc tổng thể, giao diện ioctl và dữ liệu
IV–V	Thiết kế Counter Driver; so sánh các cơ chế driver
VI–VIII	Môi trường, phương pháp và kết quả kiểm thử
IX–X	Debug, giới hạn hiện tại, kết luận và tài liệu

Quy ước bằng chứng: Mọi kết quả thực nghiệm trong báo cáo là dữ liệu đọc từ các tệp *logs/test_asgn1_output.txt* và *logs/test_counter_output.txt* thuộc gói hồ sơ nộp kèm. Các nhận xét thiết kế được đối chiếu với mã nguồn đính kèm.

II. Kiến trúc tổng thể

Hai driver đều đưa ra một đối tượng thiết bị cho userspace thông qua **Virtual File System (VFS)**. Task 1 bổ sung đường điều khiển ioctl; Task 2 công bố thống kê qua procfs bên cạnh device node. Sơ đồ dưới đây kết hợp cả hai driver trong cùng một hệ thống.



Luồng ioctl: userspace gọi ioctl(fd, cmd, arg) VFS định tuyến tới *asgn1_fops.unlocked_ioctl* đối chiếu macro command cập nhật buffer hoặc chuyển dữ liệu qua copy_to_user/copy_from_user trả mã trạng thái.

Luồng Counter: đọc */dev/counter* trả chuỗi thập phân và tăng giá trị; ghi chuỗi số hợp lệ đặt lại counter; đọc */proc/counter_info* chỉ lấy thống kê, không làm tăng counter.

III. ioctl Character Driver

3.1. Giao diện và quy ước dữ liệu

Task 1 khai báo character device **/dev/asgn1** tại major **241**, dùng *register_chrdev()*. Driver có buffer nội bộ 256 bytes, hai chế độ lưu dữ liệu và nhóm bộ đếm thống kê. Chế độ mặc định khi load là **Echo (mode 1)**.

Command / macro	Hướng	Chức năng đã triển khai
ASGN1_RESET_BUFFER **_IO('A', 1)**	Không đối số	Xóa buffer, đặt buffer_len=0 và đưa write_count/read_count về 0.
ASGN1_GET_STATS **_IOR('A', 2, stats)**	Kernel user	Trả năm trường: open_count, write_count, read_count, buffer_len, last_write_size.
ASGN1_SET_MODE **_IOW('A', 3, int)**	User kernel	mode 0 thêm tiền tố [KERNEL]; mode 1 ghi dữ liệu nguyên bản.
ASGN1_GET_VERSION **_IOR('A', 4, char[64])**	Kernel user	Trả chuỗi “asgn1_driver v1.0” trong vùng đệm 64 bytes.

3.2. Header dùng chung

```
#define ASGN1_MAGIC 'A'
#define ASGN1_RESET_BUFFER _IO(ASGN1_MAGIC, 1)
#define ASGN1_GET_STATS    _IOR(ASGN1_MAGIC, 2, struct asgn1_stats)
#define ASGN1_SET_MODE     _IOW(ASGN1_MAGIC, 3, int)
#define ASGN1_GET_VERSION  _IOR(ASGN1_MAGIC, 4, char[64])
```

File **asgn1_ioctl.h** được include trong cả kernel module và chương trình userspace **test_ioctl.c**; cách này giúp hai phía dùng chung mã lệnh và layout của cấu trúc *asgn1_stats*.

III. ioctl Character Driver (tiếp)

3.3. Luồng xử lý read / write

Hàm *asgn1_write()* kiểm tra kích thước buffer, khóa *device_mutex*, tùy mode chèn chuỗi **[KERNEL]**, dùng *copy_from_user()* để nhận dữ liệu và cập nhật write_count, last_write_size. Hàm *asgn1_read()* dùng *simple_read_from_buffer()* và tăng read_count nếu đọc được số byte dương.

3.4. Đồng bộ và bảo vệ bộ nhớ

• Một mutex tĩnh (*DEFINE_MUTEX(device_mutex)*) bao quanh thao tác trên buffer, trạng thái mode và các bộ đếm; giúp tránh hai luồng cùng chỉnh sửa dữ liệu driver tại một thời điểm.

• Các con trỏ userspace không được dereference trực tiếp trong ioctl. GET_STATS và GET_VERSION dùng *copy_to_user()*, SET_MODE dùng *copy_from_user()*; lỗi sao chép trả *-EFAULT*.

• Command không được nhận diện trả *-ENOTTY*; chế độ ngoài tập {0,1} trả *-EINVAL*.

3.5. Ý nghĩa magic number và kích thước ioctl

Magic **'A'** giúp phân biệt nhóm ioctl của driver. Các macro Linux mã hóa hướng truyền, type/magic, chỉ số lệnh và kích thước đối số. Với file header hiện tại, nhóm lệnh có chỉ số **1–4**; *_IOC_NR(cmd)* trích chỉ số, còn *_IOC_SIZE(cmd)* biểu thị số byte khai báo cho đối số.

```
/* Ví dụ đọc thông tin trường ioctl */
unsigned nr   = _IOC_NR(ASGN1_GET_STATS);
unsigned size = _IOC_SIZE(ASGN1_GET_STATS);
/* nr = 2; size = sizeof(struct asgn1_stats) */
```

Lưu ý kiểm định: Log xác nhận các thao tác hợp lệ hoạt động. Hồ sơ chưa thể hiện riêng một test command không hợp lệ để chứng minh nhánh *-ENOTTY* trong điều kiện thực tế; điều này là một đầu mục nên bổ sung nếu muốn hoàn thiện toàn bộ rubric.

IV. Counter Misc Device Driver

4.1. Mô hình triển khai

Task 2 sử dụng *struct miscdevice* kết hợp *misc_register()* và *misc_deregister()*. Thiết bị thuộc **major 10**, minor cấp phát động. Trong lượt chạy thủ công đã quan sát minor **125**; script sẽ đọc lại minor từ */proc/misc* thay vì hardcode.

Giao diện	Hành vi thực tế	API liên quan
read /dev/counter	Trả giá trị hiện tại dưới dạng chuỗi có newline, rồi tự tăng counter.	atomic_fetch_inc(); simple_read_from_buffer()
write /dev/counter	Nhận số thập phân ASCII không âm, đặt lại counter. Dữ liệu sai trả -EINVAL.	copy_from_user(); kstrtoint(); atomic_set()
read /proc/counter_info	Hiện Current counter, Total reads, Total writes.	proc_create(); seq_printf(); single_open()

4.2. Atomicity và xử lý race condition

Counter là biến chia sẻ trong kernel. Nếu nhiều luồng cùng thực hiện *value = counter; counter = value + 1* với biến *int* thường, hai lần đọc có thể lấy cùng giá trị, gây mất lượt tăng. *atomic_fetch_inc()* đọc giá trị cũ và tăng trong một thao tác nguyên tử, tránh kiểu lost update này.

Ba biến *atomic_t* độc lập được dùng cho **counter**, **total_reads**, **total_writes**. Riêng chỉ số offset **ppos* cần được kiểm tra: sau lần đọc đầu tiên trên cùng file descriptor, read() phải trả 0 ở cuối dữ liệu để các chương trình như *cat* dừng đúng.

4.3. Vòng đời tài nguyên

Trong init, driver đăng ký misc device và tạo */proc/counter_info*. Nếu bước proc_create thất bại, driver gọi *misc_deregister()* để rollback. Trong exit, gọi *proc_remove()* trước khi deregister thiết bị.

V. So sánh và quyết định thiết kế

5.1. Ba loại device driver trong Linux

Tiêu chí	Character	Block	Network
Giao diện chính	file_operations	block_device_operations và khối xử lý yêu cầu I/O	net_device_ops
Đơn vị dữ liệu	Byte stream hoặc record	Block / sector	Packet / frame
Đệm / xếp hàng	Tùy driver, buffer nội bộ	Page cache / block layer / request queue	SKB, hàng đợi TX/RX
Ví dụ nhúng	UART, cảm biến, counter ảo	eMMC/SD, thiết bị lưu trữ khối	Ethernet MAC, Wi-Fi adapter
Khi nên dùng	Thiết bị đọc ghi tuần tự, điều khiển đơn giản	Lưu trữ có địa chỉ sector	Truyền nhận khung/gói mạng

5.2. register_chrdev so với misc_register

Task 1: *register_chrdev()* phù hợp để luyện tập cách character device đăng ký major, thiết lập *file_operations* và xử lý ioctl. Nhược điểm ở bài này là major 241 được cố định và phải tạo node */dev/asgn1* trong QEMU.

Task 2: *misc_register()* giảm công việc đăng ký character device cho thiết bị nhỏ; major mặc định là 10, còn minor có thể cấp phát động. Trong rootfs hiện tại, */dev/counter* không tự xuất hiện nên script tạo node dựa trên */proc/misc*.

5.3. Hai ví dụ ra quyết định

• Nếu thiết kế counter ảo chỉ cần read/write và một entry procfs, miscdevice là lựa chọn gọn và phù hợp với Task 2.

• Nếu phát triển driver cho bộ lưu trữ eMMC có yêu cầu truy cập theo sector, block layer hợp lý hơn character driver; nếu xử lý các Ethernet frame, dùng network stack và net_device_ops.

VI. Môi trường và quy trình kiểm thử

6.1. Phần mềm và target

Hạng mục	Thông số / đường dẫn đã sử dụng
Máy host	Ubuntu Linux (kiến trúc x86-64)
Máy mô phỏng	QEMU system-arm, -M vexpress-a9, -m 512M
Kernel target	Linux 5.15.0; zImage và vexpress-v2p-ca9.dtb
Công cụ build	arm-linux-gnueabi-gcc; kbuild; .ko dạng ELF 32-bit ARM EABI5
Root filesystem	initramfs_asgn1.cpio.gz với BusyBox và module kiểm thử

6.2. Lệnh build driver ARM

```
make -C ~/embedded_lab1/kernel/linux-5.15 \
    ARCH=arm CROSS_COMPILE=arm-linux-gnueabi- \
    M=$PWD modules
```

Lệnh trên là cách được sử dụng để build Task 1 cho kernel ARM. Task 2 có Makefile đã thiết lập sẵn KDIR trỏ tới Kernel 5.15 và hỗ trợ chạy *make*. Tệp module của cả hai được kiểm tra độc lập là ELF ARM EABI5.

6.3. Lệnh khởi động QEMU

```
qemu-system-arm -M vexpress-a9 -m 512M \
  -kernel output/zImage \
  -dtb output/vexpress-v2p-ca9.dtb \
  -initrd output/initramfs_asgn1.cpio.gz \
  -append "console=ttyAMA0 rdinit=/init" -nographic
```

Các đường dẫn trong đoạn lệnh được viết tương đối so với *~/embedded_lab1/*. Trong log, hệ thống boot vào BusyBox và chạy */test_asgn1.sh*, */test_counter.sh* từ initramfs. Terminal host được ghi bằng tiện ích *script*.

VII. Kết quả kiểm thử Task 1

7.1. Bảng nghiệm thu chức năng

Trường hợp	Kết quả trong log	Đánh giá
Load module	asgn1: driver loaded, major=241	Đạt
Echo mode	Read: Hello Linux	Đạt
GET_VERSION	Version: asgn1_driver v1.0	Đạt
SET_MODE = 0	Mode set to KERNEL	Đạt
Kernel mode	Read: [KERNEL]Kernel Test	Đạt
GET_STATS	Open: 3; Write: 1; Read: 1; Buffer: 12; Last write: 12	Đạt
RESET_BUFFER	Buffer bytes after reset: 0	Đạt
Unload module	asgn1: driver unloaded; TEST FINISHED	Đạt

7.2. Trích nguyên văn từ log thực tế

```
Read: Hello Linux
Version: asgn1_driver v1.0
Mode set to KERNEL
Open count: 3
Write count: 1
Read count: 1
Buffer length: 12
Last write size: 12
Read: [KERNEL]Kernel Test
Buffer bytes after reset: 0
asgn1: driver unloaded
===== TEST FINISHED =====
```

Các số liệu thống kê đầu tiên được in **trước** thao tác reset đầu tiên. Ở lần GET_STATS thứ hai, log cho thấy open_count=6, write_count=1, read_count=1 và buffer_len=20. Việc RESET làm rỗng buffer được xác nhận độc lập bằng *cat /dev/asgn1 | wc -c* trả 0.

Nguồn: logs/test_asgn1_output.txt, phần "===== ASSIGNMENT 01 TEST =====" (22/10/2026).

VIII. Kết quả kiểm thử Task 2

8.1. Thống kê counter qua ba giai đoạn

Thời điểm	Current counter	Total reads	Total writes
Sau 3 lần đọc đầu	3	3	0
Ngay sau khi ghi 100	100	3	1
Ngay sau lệnh reset 0	0	4	2
Kết thúc kiểm thử	1	5	2

8.2. Trích log các tình huống tiêu biểu

```
Counter: 0
Counter: 1
Counter: 2
Current counter: 3
Total reads: 3
Total writes: 0
Read: 100
Read after reset: 0
PASS: negative rejected
PASS: text rejected
Current counter: 1
Total reads: 5
Total writes: 2
counter: driver unloaded
===== TEST FINISHED =====
```

Kết quả cuối có **Current counter = 1** vì phép *cat /dev/counter* sau reset đã trả về 0 và đồng thời tăng giá trị nội bộ lên 1. Hai trường hợp ghi *-5* và *abc* bị từ chối với thông báo *Invalid argument*; bộ đếm tổng số write vẫn là 2.

Nguồn: logs/test_counter_output.txt, phần "===== COUNTER DRIVER TEST =====" (22/10/2026).

IX. Debug, giới hạn và đề xuất sửa

9.1. Tình huống debug thực tế

Khi nạp *counter_driver.ko* trên hệ Linux ARM tối giản, module load thành công và */proc/misc* liệt kê **125 counter**, nhưng truy cập */dev/counter* báo *No such file or directory*. Nguyên nhân thực tế của phiên chạy là node thiết bị chưa được tự tạo trong /dev. Người thực hiện đã dùng:

```
mknod /dev/counter c 10 125
ls -l /dev/counter
cat /dev/counter    # 0, l□n ti□p theo 1, r□i 2
```

Script sau đó được thiết kế bền vững hơn: đọc minor của thiết bị từ */proc/misc* trước khi gọi *mknod*. Như vậy script không phụ thuộc vào giá trị minor 125 cố định giữa các lần khởi động.

9.2. Điểm cần hoàn thiện trước khi nộp chính thức

Nội dung	Đánh giá và hướng xử lý
MODULE_AUTHOR	Hai file .c hiện còn chuỗi “Student”; phải thay tên và MSSV.
Makefile Task 1	Target “arm” mặc định vẫn dùng KDIR của kernel host. Nên cấu hình KDIR ARM rõ ràng hoặc truyền KDIR khi build.
Test độ bền	Chưa có bằng chứng test đồng thời nhiều process, hoặc command ioctl không hợp lệ.
Counter read	Counter tăng trước khi copy_to_user hoàn tất; một lần đọc bị lỗi có thể vẫn làm tăng biến đếm.
Log chuẩn hóa	Hồ sơ có hai transcript, chưa có tệp dmesg_full.txt riêng theo cấu trúc yêu cầu.

Các điểm trên **không phủ nhận** các kết quả PASS đã ghi nhận. Chúng là giới hạn được phân biệt rõ giữa “đã test được” và “đã kiểm chứng mọi yêu cầu”.

X. Kết luận và tài liệu đối chiếu

10.1. Kết luận

Bài thực hành đã xây dựng và chạy thử hai mô-đun kernel cho ARM: một character driver mở rộng bằng bốn lệnh ioctl và một miscdevice mô phỏng bộ đếm. Hai chương trình kiểm thử shell chạy trên QEMU đến cuối mà không gặp lỗi kiểm thử, xác nhận các chức năng read/write, đổi mode, reset, statistics, xử lý giá trị không hợp lệ và cleanup theo các kịch bản hiện có.

Về mặt thiết kế, *register_chrdev()* minh họa giao diện character-device truyền thống, còn *misc_register()* tối giản việc đăng ký thiết bị nhỏ. Mutex và atomic_t được dùng ở những vị trí khác nhau phù hợp với loại trạng thái chia sẻ. Về mặt vận hành, dữ liệu log là cơ sở đánh giá thực tế, thay vì chỉ dựa vào sự kiện build ra tệp .ko.

10.2. Danh mục nguồn trong hồ sơ

Tệp nguồn / log	Nội dung sử dụng
task1_ioctl/asgn1_driver.c	Vòng đời character device, file_operations, mutex, read/write, ioctl.
task1_ioctl/asgn1_ioctl.h	Macro ioctl, magic, cấu trúc thống kê và chiều truyền dữ liệu.
task1_ioctl/test_ioctl.c	Client userspace gọi GET_VERSION, SET_MODE, GET_STATS, RESET.
task1_ioctl/test_asgn1.sh	Kịch bản Echo, Kernel, Reset và module lifecycle.
task2_misc/counter_driver.c	Miscdevice, atomic_t, procfs và validation của lệnh write.
task2_misc/test_counter.sh	Auto increment, reset, từ chối giá trị sai và gỡ module.
logs/test_asgn1_output.txt	Transcript kiểm thử Task 1, 22/10/2026.
logs/test_counter_output.txt	Transcript kiểm thử Task 2, 22/10/2026.

10.3. Kiểm tra trước khi nộp

Điền họ tên và MSSV trên trang bìa; cập nhật MODULE_AUTHOR trong source; chạy lại build khi chỉnh Makefile hoặc source; bổ sung tệp dmesg_full.txt nếu giảng viên yêu cầu đúng cấu trúc nộp. Báo cáo cần được đọc và xác nhận phù hợp với công việc thực tế của người thực hiện.

Ghi chú: tài liệu tập trung vào báo cáo phân tích & thiết kế (Task 3), không trả lời bốn câu hỏi Task 4 theo phạm vi đã chọn.